

Circular Array

A circular array simulates an unbounded, wrap-around sequence using a fixed-size block of memory. Instead of allocating new space when an array "runs out of room" at its end, a circular array reuses the freed slots at its beginning. The mechanism is modular arithmetic: for an array of capacity n , the physical index corresponding to logical position i is

$$\text{index}(i) = i \bmod n.$$

Because the remainder of division by n always lies in $\{0, 1, \dots, n - 1\}$, any integer i — no matter how large — maps back into valid bounds. This is what makes the array behave as if it wraps: incrementing past index $n - 1$ returns to index 0.

The structure is defined by two pointers, `head` and `tail`, plus a `count` of active elements. Enqueuing appends at `tail` and sets `tail = (tail + 1) mod n`; dequeuing removes from `head` and sets `head = (head + 1) mod n`. The array is full when `count == n` and empty when `count == 0` — tracking `count` explicitly avoids the ambiguity of `head == tail` meaning either "empty" or "full."

Worked example. Suppose $n = 5$, and we perform: enqueue A, B, C (`head = 0`, `tail = 3`, `count = 3`); dequeue twice (`head = 2`, `count = 1`); enqueue D, E, F. The third enqueue computes `tail = (4 + 1) mod 5 = 0`, placing F at physical index 0 — the slot vacated by A. The array's logical order (C, D, E, F) is preserved even though the elements physically wrap around the end.

Problem-solving application. Circular arrays are the standard implementation for queues, ring buffers in streaming I/O, and fixed-window algorithms (e.g., a sliding-window maximum over the last k elements, or a circular buffer feeding an audio device in real time). The key design decision is choosing n : too small forces overwrites or blocking; too large wastes memory. A common interview task is: "Implement a queue with $O(1)$ enqueue/dequeue using a fixed array" — the solution is exactly this modular-index scheme, since a naive array-shifting queue costs $O(n)$ per dequeue. Recognizing when a problem exhibits a fixed-capacity, FIFO-with-eviction pattern is the signal to reach for a circular array rather than a dynamic list.

